OWASP API Security Top 10 (2023) — APIM Mitigations

This document maps each OWASP API Security Top 10 vulnerability to specific Azure APIM policy mitigations. Each section includes the vulnerability description, real-world impact, and a ready-to-use APIM policy that mitigates it.

API1:2023 — Broken Object Level Authorization (BOLA)

The Vulnerability

An attacker modifies the object ID in a request (e.g., `/api/orders/123` → `/api/orders/456`) to access another user's data. The API doesn't verify the caller owns the requested resource.

Real-World Impact

- Access to other users' personal data, orders, medical records
- Account takeover via enumerable IDs

APIM Mitigation

Extract the user identity from the JWT and inject it into the request, forcing the backend to filter by authenticated user:

```
<inbound>
    <validate-jwt header-name="Authorization" failed-validation-httpcode="401"
                  require-signed-tokens="true">
        <openid-config url="https://login.microsoftonline.com/{tenant}/v2.0/.well-
known/openid-configuration" />
    </validate-jwt>
    <!-- Extract user ID from token and pass to backend -->
    <set-header name="X-User-Id" exists-action="override">
        <value>@{
            var jwt =
context.Request.Headers.GetValueOrDefault("Authorization","").Replace("Bearer ","");
            return jwt.AsJwt()?.Claims.GetValueOrDefault("oid", "") ?? "";
        }</value>
    </set-header>
    <!-- Prevent client from setting this header directly -->
    <check-header name="X-User-Id" failed-check-httpcode="403"
                  failed-check-error-message="Forbidden" ignore-case="false">
    </check-header>
</inbound>
```

> ***Why this works***: *Backend can use `X-User-Id` header (set by APIM, not the client) to scope queries to the authenticated user.*

API2:2023 — Broken Authentication

The Vulnerability

Weak or missing authentication allows attackers to impersonate users. Common issues: no token validation, weak passwords, missing MFA, token stuffing.

APIM Mitigation

Enforce strong JWT/OAuth 2.0 validation at the gateway:

```
<inbound>
    <validate-jwt header-name="Authorization"
                  failed-validation-httpcode="401"
                  failed-validation-error-message="Authentication required"
                  require-expiration-time="true"
                  require-signed-tokens="true"
                  clock-skew="30">
        <openid-config url="https://login.microsoftonline.com/{tenant}/v2.0/.well-
known/openid-configuration" />
        <audiences>
            <audience>{api-audience}</audience>
        </audiences>
        <issuers>
            <issuer>https://login.microsoftonline.com/{tenant}/v2.0</issuer>
        </issuers>
        <required-claims>
            <claim name="scp" match="any">
                <value>api.read</value>
            </claim>
        </required-claims>
    </validate-jwt>
</inbound>
```

API3:2023 — Broken Object Property Level Authorization

The Vulnerability

API returns excessive data (mass assignment) or allows modification of properties the user shouldn't change (e.g., setting `isAdmin: true`).

APIM Mitigation

Strip sensitive properties from responses and restrict writable fields in requests:

```
<inbound>
    <!-- Restrict request body to allowed fields only -->
    <set-body>@{
        var body = context.Request.Body.As<JObject>(preserveContent: true);
        var allowed = new[] { "name", "email", "address" };
        var filtered = new JObject();
        foreach (var field in allowed) {
            if (body[field] != null) filtered[field] = body[field];
        }
        return filtered.ToString();
```

```
    }</set-body>
</inbound>

<outbound>
    <!-- Remove sensitive fields from response -->
    <set-body>@{
        var body = context.Response.Body.As<JObject>(preserveContent: true);
        body.Remove("internalId");
        body.Remove("passwordHash");
        body.Remove("isAdmin");
        body.Remove("ssn");
        return body.ToString();
    }</set-body>
</outbound>
```

API4:2023 — Unrestricted Resource Consumption

The Vulnerability

No rate limiting or quota allows attackers to overwhelm the API with requests (DDoS), exhaust resources, or run up costs.

APIM Mitigation

Layer multiple throttling mechanisms:

```
<inbound>
    <!-- Per-IP rate limit (anonymous/unauthenticated) -->
    <rate-limit-by-key calls="20"
                       renewal-period="60"
                       counter-key="@(context.Request.IpAddress)" />

    <!-- Per-subscription rate limit (authenticated) -->
    <rate-limit-by-key calls="100"
                       renewal-period="60"
                       counter-key="@(context.Subscription?.Id ??
context.Request.IpAddress)" />

    <!-- Daily quota per subscription -->
    <quota-by-key calls="10000"
                  bandwidth="50000000"
                  renewal-period="86400"
                  counter-key="@(context.Subscription?.Id ?? context.Request.IpAddress)" />

    <!-- Max request body size (100KB) -->
    <validate-content unspecified-content-type-action="prevent"
                      max-size="102400"
                      size-exceeded-action="prevent" />
</inbound>
```

API5:2023 — Broken Function Level Authorization

The Vulnerability

Regular users can access admin endpoints (e.g., `DELETE /api/users` or `GET /api/admin/config`) because function-level authorization is missing.

APIM Mitigation

Check roles/scopes per operation:

```
<inbound>
    <validate-jwt header-name="Authorization" require-signed-tokens="true">
        <openid-config url="https://login.microsoftonline.com/{tenant}/v2.0/.well-
known/openid-configuration" />
    </validate-jwt>

    <!-- Require admin role for destructive operations -->
    <choose>
        <when condition="@(context.Request.Method == "DELETE" ||
context.Request.Url.Path.Contains("/admin"))">
            <validate-jwt header-name="Authorization">
                <required-claims>
                    <claim name="roles" match="any">
                        <value>Admin</value>
                        <value>SuperAdmin</value>
                    </claim>
                </required-claims>
            </validate-jwt>
        </when>
    </choose>
</inbound>
```

API6:2023 — Unrestricted Access to Sensitive Business Flows

The Vulnerability

Automated abuse of business features: mass purchasing, credential stuffing, content scraping, spam account creation.

APIM Mitigation

Combine rate limiting with behavioral detection:

```
<inbound>
    <!-- Aggressive rate limit on sensitive operations -->
    <choose>
        <when condition="@(context.Request.Method == "POST" &&
context.Request.Url.Path.Contains("/checkout"))">
            <rate-limit-by-key calls="5"
                               renewal-period="300"
                               counter-
```

```
key="@(context.Request.Headers.GetValueOrDefault("Authorization","anonymous"))" />
        </when>
        <when condition="@(context.Request.Method == "POST" &&
context.Request.Url.Path.Contains("/register"))">
            <rate-limit-by-key calls="3"
                               renewal-period="3600"
                               counter-key="@(context.Request.IpAddress)" />
        </when>
    </choose>

    <!-- Require additional verification header for high-value operations -->
    <check-header name="X-Captcha-Token" failed-check-httpcode="429"
                  failed-check-error-message="Additional verification required" ignore-
case="false" />
</inbound>
```

API7:2023 — Server Side Request Forgery (SSRF)

The Vulnerability

Attacker tricks the API into making requests to internal services, cloud metadata endpoints, or arbitrary URLs.

APIM Mitigation

Restrict outbound calls and block internal network access:

```
<inbound>
    <!-- Block requests containing internal URLs in the body -->
    <choose>
        <when condition="@{
            var body = context.Request.Body.As<string>(preserveContent: true);
            var blocked = new[] { "169.254.169.254", "metadata.google", "localhost",
                                  "127.0.0.1", "10.", "172.16.", "192.168.", "0.0.0.0" };
            return blocked.Any(b => body.Contains(b));
        }">
            <return-response>
                <set-status code="400" reason="Bad Request" />
                <set-body>{"error": "Request contains blocked URL patterns"}</set-body>
            </return-response>
        </when>
    </choose>
</inbound>

<backend>
    <!-- Only allow forwarding to known backend hosts -->
    <set-backend-service base-url="https://my-approved-backend.azurewebsites.net" />
</backend>
```

API8:2023 — Security Misconfiguration

The Vulnerability

Insecure defaults, unnecessary HTTP methods enabled, missing security headers, verbose error messages, CORS misconfiguration.

APIM Mitigation

Enforce secure configuration at the gateway:

```
<inbound>
    <!-- Block unnecessary HTTP methods -->
    <choose>
        <when condition="@(new[]
{"TRACE","OPTIONS","CONNECT","PATCH"}.Contains(context.Request.Method))">
            <return-response>
                <set-status code="405" reason="Method Not Allowed" />
            </return-response>
        </when>
    </choose>

    <!-- Enforce HTTPS only -->
    <choose>
        <when condition="@(context.Request.OriginalUrl.Scheme != "https")">
            <return-response>
                <set-status code="403" reason="HTTPS Required" />
                <set-body>{"error": "HTTPS is required for all API calls"}</set-body>
            </return-response>
        </when>
    </choose>
</inbound>

<outbound>
    <!-- Security headers -->
    <set-header name="X-Content-Type-Options" exists-action="override">
<value>nosniff</value></set-header>
    <set-header name="X-Frame-Options" exists-action="override"><value>DENY</value></set-
header>
    <set-header name="Strict-Transport-Security" exists-action="override">
        <value>max-age=31536000; includeSubDomains</value>
    </set-header>
    <set-header name="Cache-Control" exists-action="override"><value>no-store</value></set-
header>

    <!-- Remove server fingerprinting headers -->
    <set-header name="X-Powered-By" exists-action="delete" />
    <set-header name="Server" exists-action="delete" />
    <set-header name="X-AspNet-Version" exists-action="delete" />
</outbound>
```

API9:2023 — Improper Inventory Management

The Vulnerability

Old API versions, deprecated endpoints, or undocumented APIs remain accessible, providing attack surface.

APIM Mitigation

Use APIM's versioning and revision features with policies:

```
<inbound>
    <!-- Enforce minimum API version -->
    <choose>
        <when condition="@{
            var version = context.Request.Headers.GetValueOrDefault("api-version",
                          context.Request.Url.Query.GetValueOrDefault("api-version", ""));
            var deprecated = new[] { "v1", "2020-01-01", "2021-06-01" };
            return deprecated.Contains(version);
        }">
            <return-response>
                <set-status code="410" reason="Gone" />
                <set-body>@{
                    return new JObject(
                        new JProperty("error", "This API version is deprecated"),
                        new JProperty("message", "Please upgrade to the latest version"),
                        new JProperty("docs", "https://developer.example.com/migration-
guide")
                    ).ToString();
                }</set-body>
            </return-response>
        </when>
    </choose>
</inbound>
```

API10:2023 — Unsafe Consumption of APIs

The Vulnerability

Your API blindly trusts data from third-party APIs without validation, leading to injection, data corruption, or information disclosure.

APIM Mitigation

Validate and sanitize responses from external APIs:

```
<outbound>
    <!-- Validate third-party response -->
    <choose>
        <when condition="@(context.Response.StatusCode != 200)">
            <return-response>
                <set-status code="502" reason="Bad Gateway" />
                <set-body>{"error": "Upstream service returned an error"}</set-body>
            </return-response>
```

```
        </when>
    </choose>

    <!-- Sanitize response from third-party -->
    <set-body>@{
        var body = context.Response.Body.As<JObject>(preserveContent: true);
        // Remove any unexpected fields from third-party response
        var allowed = new[] { "id", "name", "status", "data" };
        var sanitized = new JObject();
        foreach (var field in allowed) {
            if (body[field] != null) sanitized[field] = body[field];
        }
        return sanitized.ToString();
    }</set-body>
</outbound>
```

Summary Table

OWASP ID	Vulnerability	Key APIM Policies
API1	Broken Object Level Auth	`validate-jwt` + `set-header` (inject user ID)
API2	Broken Authentication	`validate-jwt` with full claims validation
API3	Property Level Auth	`set-body` (filter fields in/out)
API4	Unrestricted Consumption	`rate-limit-by-key` + `quota-by-key` + `validate-content`
API5	Function Level Auth	`validate-jwt` + `choose` (role-based per operation)
API6	Sensitive Business Flows	`rate-limit-by-key` + `check-header` (captcha)
API7	SSRF	`choose` (block internal URLs) + `set-backend-service`
API8	Security Misconfiguration	`set-header` (security headers) + method blocking
API9	Inventory Management	`choose` (version deprecation)
API10	Unsafe 3rd Party Consumption	`set-body` (sanitize upstream response)

Previous: ← Policy Pipeline | Next: Security Automation →